Name: Sanjog Poojary

Roll no : 38

Experiment 6

Aim

Experiment to implement any one of the clustering algorithms using Rapid Miner and Python.

Theory

- Discover the natural grouping or structure in unlabelled data without predefined categories.
- Data points are assigned to clusters based on similarity or distance measures.
- Uses Euclidean distance, cosine similarity or other metrics depending on data type and clustering method.

Types of Clustering 1.Hard Clustering Hard clustering assigns each data point to exactly one cluster. A data point cannot belong to multiple clusters, making the grouping clear and easy to interpret.

- Each data point belongs to only one cluster
- No overlap between clusters
- Simple and easy to interpret 2.Soft Clustering Soft clustering allows a data point to belong to multiple clusters with different probabilities. Instead of assigning a strict cluster, it gives a degree of membership to each cluster. Example A data point may belong 70% to Cluster 1 and 30% to Cluster 2, indicating that it shares characteristics with both groups.

Use Cases

- Overlapping class boundaries: Useful when data points cannot be clearly separated into distinct groups.
- Customer personas: Helps represent customers who share traits with multiple behavioral groups.
- Medical diagnosis: Patients may show symptoms related to multiple conditions.

Benefits

- Captures ambiguity: Represents uncertainty when cluster boundaries are not clear.
- Models gradual transitions: Allows smooth transitions between clusters instead of strict separation.

K-Means Clustering: K-Means Clustering groups similar data points into clusters without needing labeled data. It is used to uncover hidden patterns when the goal is to organize data based on similarity.

- Helps identify natural groupings in unlabeled datasets
- Works by grouping points based on distance to cluster centers
- Commonly used in customer segmentation, image compression and pattern discovery
- Useful when you need structure from raw, unorganized data. Uses of K-Means Clustering K-Means is popular in a wide variety of applications due to its simplicity, efficiency and effectiveness. Here’s why it is widely used:

- Data Segmentation: One of the most common uses of K-Means is segmenting data into distinct groups. For example, businesses use K-Means to group customers based on behavior, such as purchasing patterns or website interaction.
- Image Compression: K-Means can be used to reduce the complexity of images by grouping similar pixels into clusters, effectively compressing the image. This is useful for image storage and processing.
- Anomaly Detection: K-Means can be applied to detect anomalies or outliers by identifying data points that do not belong to any of the clusters.
- Document Clustering: In natural language processing (NLP), K-Means is used to group similar documents or articles together. It's often used in applications like recommendation systems or news categorization.
- Organizing Large Datasets: When dealing with large datasets, K-Means can help in organizing the data into smaller, more manageable chunks based on similarities, improving the efficiency of data analysis.

Dataset Overview

- **Title:** JioHotstar Preprocessed Dataset
- **Size:** 33,171 records and 24 columns (shape (33171, 24)).
- **Columns used for clustering:** episode_duration_min, pacing_score, hook_strength, visual_intensity, avg_watch_percentage, pause_count, rewind_count, skip_intro, cognitive_load, night_watch_safe.
- **Excluded:** drop_off and drop_off_probability, because clustering is unsupervised.

Implementation (Python)

Step 1: Load the dataset

```
from google.colab import files
uploaded = files.upload()
filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("Dataset Shape:", df.shape)
print("\nDataset Columns:")
print(df.columns.tolist())
print("\nFirst 5 Records:")
display(df.head())
```

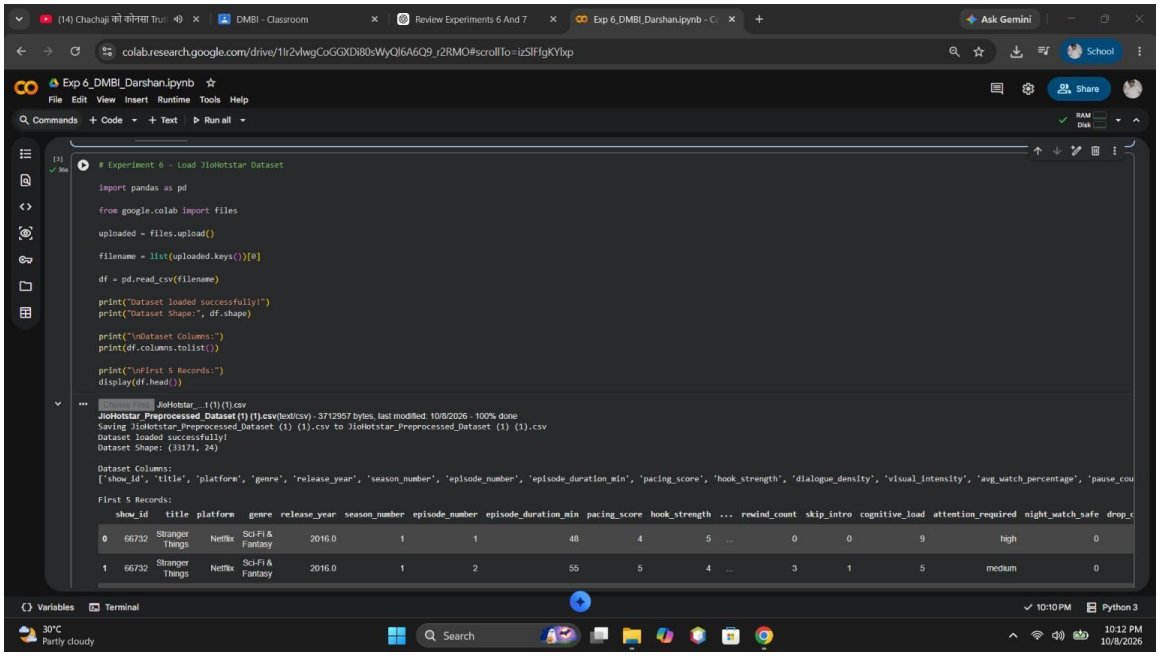


Figure 1: Dataset loaded in Google Colab with shape, columns and first records

Inference: The JioHotstar dataset was successfully loaded into Python using the pandas library. The dataset contains information about episode characteristics and viewer behaviour. Checking the number of rows and columns helps us understand the size and structure of the data before applying clustering.

Step 2: Select features

```
features = [
    'episode_duration_min',
    'pacing_score',
    'hook_strength',
    'visual_intensity',
    'avg_watch_percentage',
    'pause_count',
    'rewind_count',
    'skip_intro',
    'cognitive_load',
    'night_watch_safe'
]

X = df[features]
print("Selected Features:")
print(features)
print("\nShape of selected data:", X.shape)
print("\nFirst 5 rows of selected features:")
display(X.head())
```

```
# Experiment 6 - Select features for K-Means clustering

features = [
    'episode_duration_min',
    'pacing_score',
    'hook_strength',
    'visual_intensity',
    'avg_watch_percentage',
    'pause_count',
    'rewind_count',
    'skip_intro',
    'cognitive_load',
    'night_watch_safe'
]

X = df[features]

print("Selected Features:")
print(features)

print("\nShape of selected data:", X.shape)

print("\nFirst 5 rows of selected features:")
display(X.head())
```

```
Selected Features:
['episode_duration_min', 'pacing_score', 'hook_strength', 'visual_intensity', 'avg_watch_percentage', 'pause_count', 'rewind_count', 'skip_intro', 'cognitive_load', 'night_watch_safe']

Shape of selected data: (33171, 10)

First 5 rows of selected features:
```

	episode_duration_min	pacing_score	hook_strength	visual_intensity	avg_watch_percentage	pause_count	rewind_count	skip_intro	cognitive_load	night_watch_safe
0	48	4	5	5	39	3	0	0	9	0
1	55	5	4	8	55	3	3	1	5	0
2	51	4	8	7	46	4	2	0	9	0
3	50	4	7	3	50	4	1	0	7	0
4	52	4	3	4	35	3	0	1	7	0

Figure 2: Selected clustering features and their first five rows

Inference: Relevant features such as average watch percentage, episode duration, hook strength, pacing score, pause count, rewind count, and cognitive load were selected for clustering. The features help identify similarities in viewing behaviour. The target-related columns drop_off and drop_off_probability were excluded because this experiment uses unsupervised learning.

Step 3: Standardize the features

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(X_scaled, columns=features)

print("Standardization completed successfully!")
print("\nStandardized data shape:", X_scaled.shape)
print("\nFirst 5 rows of standardized data:")
display(X_scaled.head())
```

```
[5]
2s
# Experiment 6 - Standardize the selected features

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(X_scaled, columns=features)

print("Standardization completed successfully!")

print("\nStandardized data shape:", X_scaled.shape)

print("\nFirst 5 rows of standardized data:")
display(X_scaled.head())
```

```
Standardization completed successfully!

Standardized data shape: (33171, 10)

First 5 rows of standardized data:
```

	episode_duration_min	pacing_score	hook_strength	visual_intensity	avg_watch_percentage	pause_count	rewind_count	skip_intro	cognitive_load	night_watch_safe
0	0.451511	-1.136141	-0.303547	-0.506150	-1.415761	-0.055129	-1.293941	-0.986614	2.120815	-0.127058
1	0.757196	-0.323397	-0.881557	0.998107	-0.143579	-0.055129	0.643583	1.013568	-0.904614	-0.127058
2	0.582519	-1.136141	1.430485	0.496688	-0.859182	0.480515	-0.002259	-0.986614	2.120815	-0.127058
3	0.538850	-1.136141	0.852475	1.500989	-0.541136	0.480515	0.648100	0.986614	0.608100	-0.127058

Figure 3: Standardized features using StandardScaler

Inference: The selected features were standardized using StandardScaler. This converts the features to a comparable scale, generally with a mean of approximately zero and a standard deviation of approximately one. Standardization is important because K-Means uses distance calculations, and features with larger numerical values could otherwise dominate the clustering process.

Step 4: Elbow method to choose K

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

inertia = []
K_values = range(2, 11)

for k in K_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(K_values, inertia, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method for JioHotstar Dataset")
plt.xticks(K_values)
plt.grid(True)
plt.show()
```

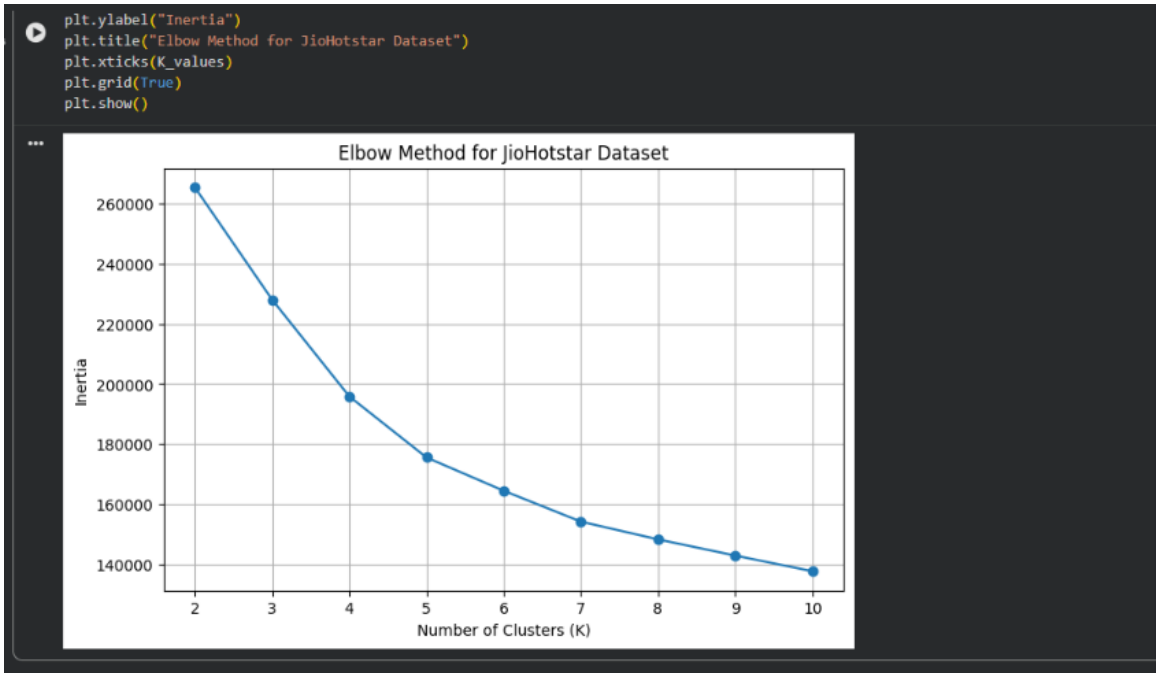


Figure 4: Elbow method: inertia against number of clusters K

Inference: The Elbow Method was used to compare the inertia values for different numbers of clusters (K). Inertia measures the sum of squared distances between data points and their assigned cluster centroids. The elbow plot helps identify a suitable number of clusters. In this experiment, K = 4 was selected for clustering.

Step 5: Apply K-Means (K = 4)

```
kmeans = KMeans(
    n_clusters=4,
    random_state=42,
    n_init=10
)

cluster_labels = kmeans.fit_predict(X_scaled)

# Add cluster labels to the original dataset
df['Cluster'] = cluster_labels

print("K-Means clustering completed successfully!")
print("\nNumber of records in each cluster:")
print(df['Cluster'].value_counts().sort_index())
```

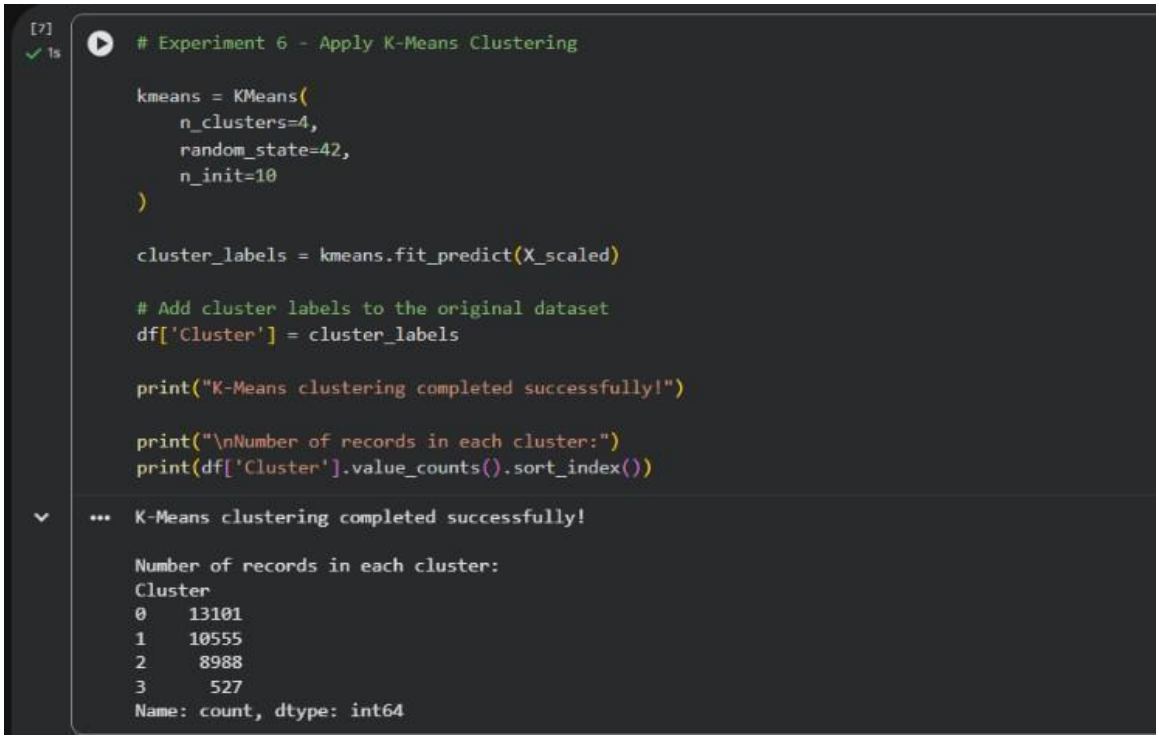


Figure 5: K-Means applied with K = 4 and the number of records per cluster

Inference: The K-Means algorithm was applied with four clusters using n_clusters=4, random_state=42, and n_init=10. The algorithm grouped records based on similarities in their selected features. Each record was assigned a cluster label from 0 to 3, allowing different viewer-behaviour patterns to be studied.

Step 6: Clustering evaluation

```
from sklearn.metrics import silhouette_score, davies_bouldin_score

# Silhouette Score
silhouette = silhouette_score(X_scaled, cluster_labels)

# Inertia
inertia_value = kmeans.inertia_

# Davies-Bouldin Score
davies_bouldin = davies_bouldin_score(X_scaled, cluster_labels)
```

```
print("Clustering Evaluation Metrics")
print("------------------------------")
print("Number of Clusters (K):", 4)
print("Silhouette Score:", silhouette)
print("Inertia:", inertia_value)
print("Davies-Bouldin Score:", davies_bouldin)
```

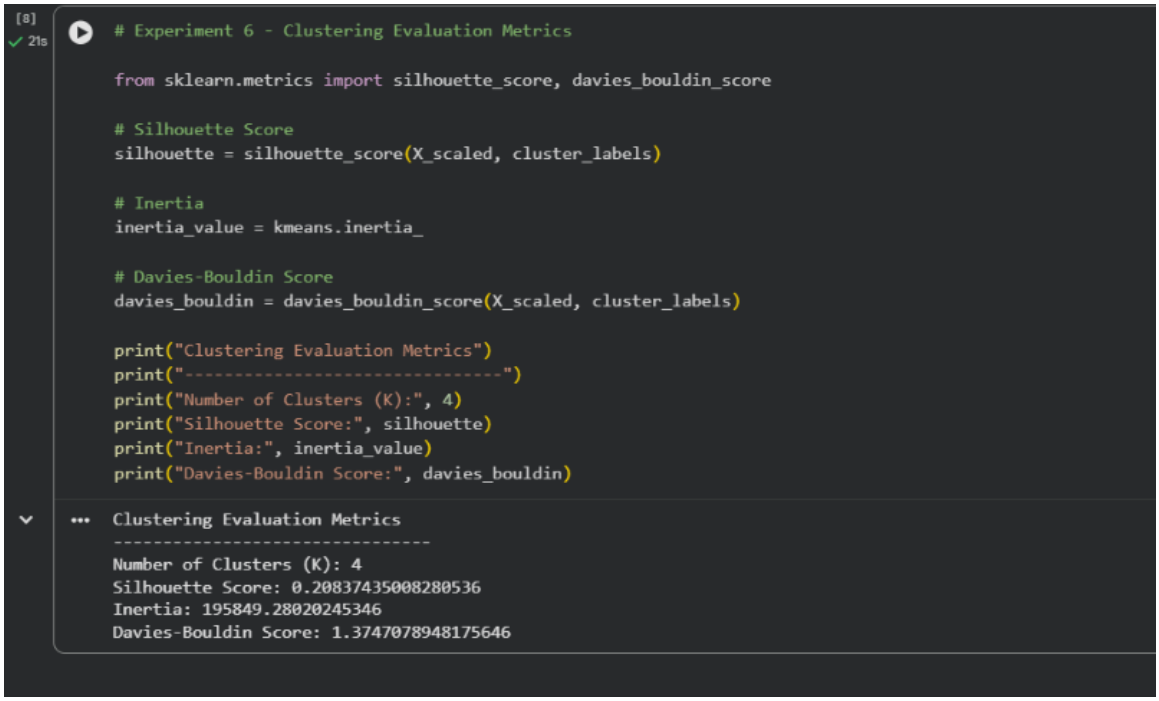


Figure 6: Silhouette score, inertia and Davies-Bouldin score

Inference: Clustering evaluation measures were used to assess the quality of the clusters. Inertia indicates how closely records are grouped around their centroids. The Silhouette Score evaluates how well-separated the clusters are, while the Davies–Bouldin Score measures similarity between clusters. These measures help assess the clustering result; the actual scores should be reported only if they are visible in the output.

Step 7: Visualize the clusters

```
plt.figure(figsize=(10, 6))

plt.scatter(
    df['avg_watch_percentage'],
    df['cognitive_load'],
    c=df['Cluster'],
    cmap='viridis',
    alpha=0.6
)

plt.xlabel("Average Watch Percentage")
plt.ylabel("Cognitive Load")
plt.title("K-Means Clustering of JioHotstar Data")
plt.colorbar(label="Cluster")
plt.grid(True)
plt.show()
```

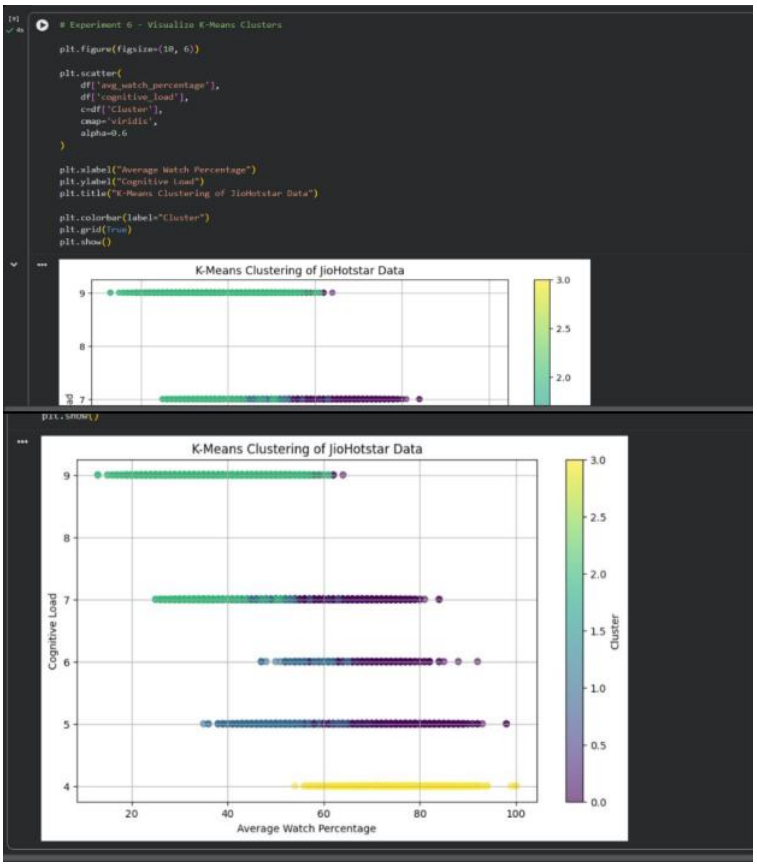


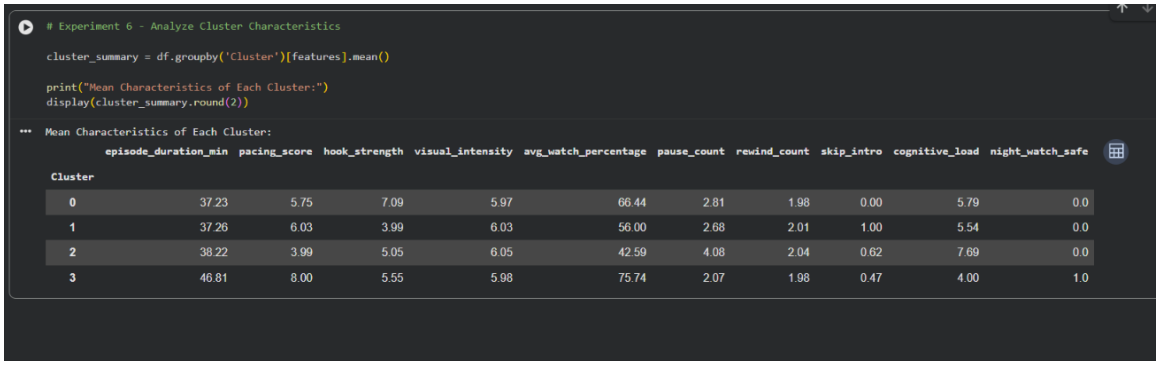
Figure 7: Scatter plot of average watch percentage against cognitive load, coloured by cluster

Inference: A scatter plot was created using average watch percentage on the X-axis and cognitive load on the Y-axis. Different colours represent the four clusters formed by K-Means. The visualization helps compare viewer-engagement patterns and observe how records are distributed among the clusters.

Step 8: Cluster characteristics

```
cluster_summary = df.groupby('Cluster')[features].mean()

print("Mean Characteristics of Each Cluster:")
display(cluster_summary.round(2))
```



Mean Characteristics of Each Cluster:

Cluster	episode_duration_min	pacing_score	hook_strength	visual_intensity	avg_watch_percentage	pause_count	rewind_count	skip_intro	cognitive_load	night_watch_safe
0	37.23	5.75	7.09	5.97	66.44	2.81	1.98	0.00	5.79	0.0
1	37.26	6.03	3.99	6.03	56.00	2.68	2.01	1.00	5.54	0.0
2	38.22	3.99	5.05	6.05	42.59	4.08	2.04	0.62	7.69	0.0
3	46.81	8.00	5.55	5.98	75.74	2.07	1.98	0.47	4.00	1.0

Figure 8: Mean feature values of each cluster

Inference: Cluster-wise averages were calculated to understand the characteristics of each group. Cluster 0 showed good engagement, Cluster 1 showed moderate engagement with intro skipping, Cluster 2 showed lower watch percentage and higher cognitive load, and Cluster 3 showed the highest average watch percentage with high pacing and lower cognitive load. These results demonstrate that K-Means can identify different viewing-behaviour patterns in the JioHotstar dataset.

Results

Table 1: Cluster sizes and mean characteristics (K = 4)

Cluster	Records	Avg watch %	Pacing score	Hook strength	Cognitive load	Night-watch safe
0	13,101	66.44	5.75	7.09	5.79	0.00
1	10,555	56.00	6.03	3.99	5.54	0.00
2	8,988	42.59	3.99	5.05	7.69	0.00
3	527	75.74	8.00	5.55	4.00	1.00

Table 2: Clustering evaluation metrics

Metric	Value
Silhouette score	0.2084
Inertia	195,849.28
Davies-Bouldin score	1.3747

Conclusion

In this experiment, K-Means clustering was successfully performed on the JioHotstar dataset using Python. The Elbow Method was used to select four clusters, and the clusters were evaluated using different performance measures. The results helped identify different viewer-behaviour patterns based on watch percentage, cognitive load, pacing score, and hook strength. Thus, the experiment demonstrated how unsupervised machine learning can be used to analyze viewer engagement and understand content-viewing patterns.

GitHub Folder: https://github.com/Sanjog-Poojary/DMBI-Lab/tree/main/Exp6_KMeans_Clustering